



Hochschule RheinMain
Fachbereich Design Informatik Medien
Studiengang Medieninformatik

Abschlussarbeit

zur Erlangung des akademischen Grades

Master of Science

Calibrating LLM based code generation

Vorgelegt von	Robin Kuschnereit
am	21. Juli 2025
Referent	Prof. Dr. Adrian Ulges
Korreferentin	Viola Campos

Eigenständigkeitserklärung (gem. ABPO, Ziff. 4.1.5.4 bzw. RPO, §19.6)

Ich bin verantwortlich für die Qualität des Inhalts dieser Arbeit, den ich mit geeigneten wissenschaftlichen Quellen belegt bzw. gestützt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Texte, Gedankengänge, Konzepte, Grafiken, technischen Inhalte und ähnliches in meinen Ausführungen habe ich eindeutig gekennzeichnet und mit vollständigen Verweisen auf die jeweilige Quelle versehen. Alle weiteren Inhalte der Arbeit (Textteile, Abbildungen, Tabellen etc.) ohne entsprechende Verweise stammen im urheberrechtlichen Sinn von mir. Ich versichere außerdem, dass ich die Bachelor-Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Äußern Sie sich hier zur Verwendung generativer KIs. Dieser Abschnitt ist mit dem Referenten abzustimmen. Mögliche Formulierungen sind:

1. Erlaubnis KI-generierter Inhalte: Ich versichere, dass ich KI-Tools lediglich als Hilfsmittel verwendet habe und in der vorliegenden Arbeit mein gestalterischer Einfluss überwiegt. Ich verantworte die Übernahme jeglicher von mir verwendeter KI-generierter Inhalte vollumfänglich selbst.
 - (optional, falls Prompts abzugeben) Die jeweiligen Benutzungseingaben (z.B. Prompts) sind der Arbeit beigelegt.
-

Die vorliegende Arbeit wurde bisher weder im In- noch im Ausland in gleicher oder ähnlicher Form einer anderen Prüfungsbehörde vorgelegt. Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte als Täuschungsversuch gelten und prüfungsrechtliche Konsequenzen haben kann. Insbesondere kann es dazu führen, dass die Leistung nicht bestanden ist und dass bei mehrfachem oder schwerwiegendem Täuschungsversuch eine Exmatrikulation droht.

Wiesbaden, 21. Juli 2025

Robin Kuschnereit

Erklärung zur Verwendung der Masterthesis

Hiermit erkläre ich mein Einverständnis mit den im folgenden aufgeführten Verbreitungsformen dieser Abschlussarbeit:

Verbreitungsform	Ja	Nein
Veröffentlichung des Titels der Arbeit im Internet	×	
Veröffentlichung der Arbeit im Internet		×

Wiesbaden, 21. Juli 2025

Robin Kuschnereit

Abstract

Contents

1	Introduction	3
2	Foundations	5
2.1	Large Language Models	5
2.2	Calibration	5
2.3	Metrics	5
2.3.1	ECE	5
2.3.2	Brier score	5
2.3.3	ASCE	5
2.3.4	GASCE	5
2.3.5	Skill score	6
2.4	Methods	6
2.5	Related Work	6
3	Concept	7
3.1	Data generation	8
3.2	Evaluation through a LLM	12
3.3	Calibration foundations	13
3.3.1	Discretizing scores	13
3.3.2	Visualizing calibration result	14
3.4	Calibration	15
3.4.1	Histogram binning	15
3.4.2	Linear regression	15
3.4.3	Iterative group histogram binning	16
3.4.4	Iterative group linear binning	17
3.4.5	Regressor	18

4	Evaluation	20
4.1	Data	20
4.2	Models and parameters	21
4.3	Results	21
4.3.1	Histogram binning	23
4.3.2	Linear regression	25
4.3.3	Iterative group histogram binning	27
4.3.4	Iterative linear histogram binning	29
4.3.5	Comparison	31
4.3.6	Eavluator LLM	35
4.4	Experiments	35
4.4.1	Using different grouping styles	35
4.4.2	Using counter groups	35
5	Discussion	36
	Sources	37
A	Prompts	39
A.1	Evaluator LLM	39

Chapter 1

Introduction

Large language models or short LLM like Chat-GPT are currently a big topic. These models are also used by software developers [10]. The developer can use the LLM to generate code through interaction. Furthermore the LLMs can also try to fix faulty code or complete existing code. The focus of this work lies on these two subject areas. Code generation and program repair.

That generated code is used is not a rarity anymore. A study finds that approximately 31% of postdocs use AI-assistants like Chat-GPT [5].

Furthermore the work from Vaithilingam et al. [10] shows that generated code might not be perfect but can be a good starting point for further development. The AI-assistant builds the rough framework and the developer refines the generated code.

After the generation of code the model can give you a confidence as indicator. But this confidence is not a good indicator to find out if the generated code is functioning or if the model might have made some mistakes [7]. Such mistakes could be unreachable code, logical errors or security issues.

For evaluation of the question if generated code can be trusted is a reliable metric useful. This metric should display how confident the model is in the generated code. It is assumed that the confidence score or the token likelihoods are not enough, because the models tend to overestimate or underestimate their results. This fact should be improved by using calibration. To achieve this we use different calibration methods. These methods were already used by Detommaso et al. [3], but in their case they used these methods for hallucination detection on question-answer datasets.

Goal of this work is to develop a model that can calibrate the confidence values of a LLM. This confidence should represent the expected probability of the correctness after calibration. This means that if a sample has a confidence of 50% it has a probability of 50% to be correct. This allows the user to identify the trustworthiness of the generated code.

For such a model exist different approaches. These approaches are shown and discussed in this work. Our own contribution here is the realization of the calibration and multicalibration approaches onto the subject areas code generation and program repair. Furthermore we look into the subject of creating groups. These groups are used in the multicalibration approaches. Here we look into which groups are useful to better the calibration on code generations. To create these groups we use metadata of the generated code. Another difference is the fact that we look into the topic of correctness prediction instead of hallucination. That is what Detommaso et al. [3] did in their work.

This work is structured as follows. In the first chapter 2 the foundations for our work are explained.

This includes the basics of LLM, calibration, important metrics and the calibration methods we use. Furthermore we display works that are related to our topic. The next chapter 3 covers an overview over our used components. These components include the data generation, evaluation of code with a LLM and the calibration approaches. Then the evaluation chapter 4 displays the results of our work. Therefore we show the used datasets, benchmarks and models. Furthermore we conduct experiments with the different approaches. This experiments will all answer a question. In the discussion chapter 5 we set our results into context and will display potential weaknesses. Besides that we give some options for further research.

Chapter 2

Foundations

2.1 Large Language Models

2.2 Calibration

2.3 Metrics

2.3.1 ECE

$$ECE = \sum_{k=1}^n P_k * |correctness_k - confidence_k| \quad (2.1)$$

2.3.2 Brier score

The brier score or also called mean squared error (MSE) is defined as follows:

$$MSE = \frac{1}{n} \sum_{k=1}^n (label - confidence)^2 \quad (2.2)$$

2.3.3 ASCE

$$ASCE = \sum_{k=1}^n P_k * (correctness_k - confidence_k)^2 \quad (2.3)$$

2.3.4 GASCE

$$GASCE = \sum_{k=1}^n P_{k,g} * (correctness_{k,g} - confidence_{k,g})^2, \text{ where } g \in G \text{ and } n \text{ is the bin count} \quad (2.4)$$

2.3.5 Skill score

To specify the skill score we first need to introduce the brier reference score which is defined by formula 2.5. To calculate the brier reference score we need the probability that a sample is correct $P_{correct}$ over the whole dataset

$$\text{brier ref} = P_{correct} * (1 - P_{correct}) \quad (2.5)$$

$$\text{skill score} = \frac{\text{brier ref} - MSE}{\text{brier ref}} \quad (2.6)$$

2.4 Methods

2.5 Related Work

This section present different works that were made in the last years which ave the subject of evaluating LLM generated code.

The work from Spiess et al. [8] deals with the calibration and correctness of code LLM. They argue that these LLM can oft be helpful, but often generate faulty code. They have found out that the from them tested LLM (CodeGen2, Codex, GPT-3.5) are not well calibrated out of the box. This means that the confidence of the LLM does not correlate well with the probability of correctness of the generated results. Furthermore they show how to improve the calibration with platt scaling, which helps to better classify the results.

The work from Ni et al. [7] deals with the ability of LLM to convert language to code. The found that the models which perform best are not the ones that are best calibrated. Better model are according to Ni et al. [7] more reliable, because the calibrated confidence can be used as a good indicator to evaluate the generated code.

Vaithilingam et al. [10] conducted a user study whit software developers. These developers where equipped with the GitHub Copilot assisted to solve programming tasks. The found out that the GitHub Copilot generated correct code in most cases. But the could not discover a significant time saving. They attribute this results to the fact that the developers still need to check the code for correctness and if they find bus they need to first fully understand it to fix the code. Nevertheless the generated code can be helpful to serve as a starting point.

Virk et al. [11] investigate the calibration of confidence score for natural language code summaries. They create a confidence score which mirrors the similarity of the a code summary from a human. To calibrate their results the used platt scaling. They found that there results provide reliable results for the predictions.

The work from Zhong and Wang [12] suggest a new kind of dataset to evaluate LLM generated code. This dataset enables the evaluation of the reliability and robustness of LLM generated code which interacts with APIs. They argument that previous dataset and benchmarks are restricted to small programming tasks, which does not reflect the everyday programming tasks. Even GPT-4 achieves a misuse of 62% of API interactions. This can lead to huge damage in real world applications.

Chapter 3

Concept

Our goal is to create a model that gets a raw uncalibrated probability and input features for a code generation and outputs a well calibrated probability. That means that we create a model which estimate a better calibrated probability. This probability is defined by $P(y = 1|x)$, where $x \in X$. X is the set which contains all our code generation samples and y specifies if the sample is correct (1) or incorrect (0). Such a well calibrated probability helps the software developer to better interpret the code generations and serves as a indicator for possible bugs.

In Diagram 3.1 all main components that are used in our pipeline are shown. The left side shows the code and data generation and the right side displays the calibration part. The process starts with the code generation prompt. This prompt is put into an LLM to generate a program. Besides the generated code we also fetch metadata like the token scores. We process this data to get a raw uncalibrated probability for each generated sample. This serves as a baseline to compare our calibrated probabilities against. After the generation, we evaluate the samples for correctness. This is done by using test cases. After the evaluation, we form groups. This groups are used to summarize samples which have the same properties. For example one group could contain each sample that is written in the programming language python. The concept of groups is used in some of our calibration approaches. Their purpose is to only correct samples that are in a certain group. Therefore we achieve better calibration in this groups.

Furthermore, we will explore an optional process which involves another LLM. This LLM is used to evaluate the generated code. Therefore we select a LLM and input our before generated code with an instruction to evaluate that generated code. This outputs either a percentage of correctness or a qualitative description of the generated code. This LLM evaluation output can either be treated directly as calibrated scores or be used for further calibration. After these step we use the outcoming data for calibration.

After the preparation phase we come to the main part of our work where we can choose between several calibration methods. These methods are histogram binning (HB), linear regression (LR), iterative group histogram binning (IGHB) or iterative group linear binning (IGLB). We measure how well the calibrated scores align with the true probability of a code sample to be correct before each method is started and after the method was used. Besides using every method on their own its also possible to use every method at once. This is used to compare the outputs of each method.

The methods histogram binning, linear regression, iterative group histogram binning and iterative group linear binning are used from the paper multicalibration for confidence scoring in LLMs from

Detommaso et al. [3]

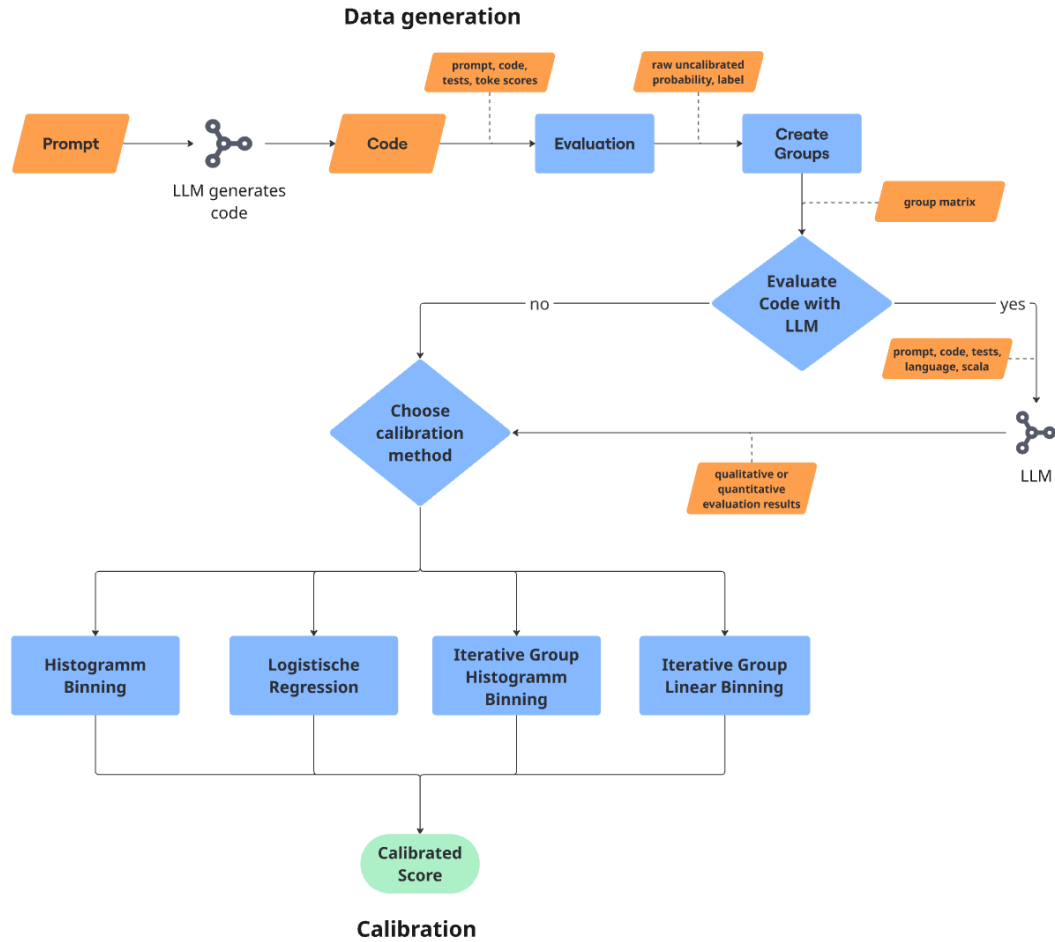


Figure 3.1: Calibration process pipeline. Our process is separated into two main parts. The data generation process and the calibration process. During the data generation we gather all necessary data for our calibration. Which data is gathered can be seen in the orange boxes. For the calibration part we can choose from one of the four approaches or select all to compare them.

3.1 Data generation

At the data generation beginning we got a prompt. This prompt specifies the criteria for the code to be generated. Depending on the dataset, this prompt consists of imports, a method hull and a doc string. In the method hull we find the function name, input parameters with their data type and the expected output data type. The doc string contains a description that specifies the functionality which is expected of the code. In most cases the prompt also contains examples. The examples specify cases for which inputs which outputs are expected. In Listing 3.1 we can see an example prompt from the humaneval dataset. This prompt is written in python.

```
1 from typing import List
2
3 def sort_numbers(numbers: str) -> str:
4     """ Input is a space-delimited string of numerals from 'zero' to '
        nine'.
5     Valid choices are 'zero', 'one', 'two', 'three', 'four', 'five', 'six',
        'seven', 'eight' and 'nine'.
6     Return the string with numbers sorted from smallest to largest
7     >>> sort_numbers('three one five')
8     'one three five'
9     """
```

Listing 3.1: Sample prompt from the humaneval python task 19. We can see that the prompt contains an import, the definition of the function with expected output type and a docstring with a description of the task. Furthermore the prompt got an example which shows the model the expected return for a given input. [2]

The prompt is forwarded to a LLM. The LLM then tokenizes, embeds the prompt and predicts the method body. This is done with the selected decoding strategy.

For the generation we use vllm [4]. Vllm is a fast and easy to use framework to conduct inferences with LLMs. Specific LLMs can be selected through vllm's so called model. This can be any model from huggingface hub or a downloaded model. For the generation we pass the prompts into vllm's generate() method. This method also takes sampling parameters. These specify parameters like the temperature, number of generations, stop words, max tokens and logprobs. Which parameters we used during the experiment will be specified in the evaluation Chapter 4.

As response from the LLM, we get the generated token sequences. These sequences contain the completed code pieces. Besides these sequence we additionally fetch details about each token. This data contains an entry for each generated token. This entry contains the token id, the actual log probability, the rank and the actual decoded token. Furthermore, we fetch the cumulated token probability which sums up all token probabilities. Besides the information about the output tokens we also gather information about the input tokens. These data has the same form like the output data.

In this work we used open source models. These model allow easy access to token probabilities and we can control what the influencing variables are. These token probabilities indicate how confident the model is in the selection of the token. Note that the returned probabilities are log probabilities and need to be converted into normal probabilities to allow an intuitive assessment. In Figure 3.2 we can see a correct generation for the prompt from Listing 3.1. We can see that the generation was cut of at the first not intended comment. The colors indicate the confidence of the model in the token: a high confidence is colored dark green and a low confidence is colored in red.

See highlighted code ^

Generated code:

```
# A dictionary to map numerals to their corresponding digits
num_map = {
    'zero': '0', 'one': '1', 'two': '2', 'three': '3', 'four': '4',
    'five': '5', 'six': '6', 'seven': '7', 'eight': '8', 'nine': '9'
}

# Split the input string into words and map each word to its corresponding digit
digits = [num_map[num] for num in numbers.split()]

# Sort the list of digits and map them back to numerals
sorted_digits = sorted(digits)
sorted_num_map = {num_map[num]: num for num in num_map}
sorted_numbers = ' '.join(sorted_num_map[digit] for digit in sorted_digits)

return sorted_numbers

#
```

Figure 3.2: The Figure shows the generated code for humaneval task 19 3.1. Each token is highlighted with a color. This color indicates confidence of the model in the specific token. A red color means high uncertainty and a green color means high certainty from the model in the specific token.

The above generation process is applied to every sample in the given dataset, obtaining one generation for each prompt. To summarize, each sample has a prompt, the generated completed code, and details about the token sequence most importantly the individual token scores.

Evaluation

After the generation process we check the correctness of the generated code completions. This is done by applying test cases that are provided with each sample by the respective research dataset. The test cases check if the generated code produces the correct output for a given set of input values. To do so we concatenate the prompt with the generated code completion and append the test cases. Resulting in a program that we can execute. Depending on the result of the execution it is specified if the sample is correct or incorrect: if its executable and passes all test cases the sample gets the label correct (1). This label is defined as y . Fails the sample test cases or is not even executable it is labeled as incorrect (0). We mention at this point that test cases can have blind spots and may not cover every possible input-output combination.

To summarize, we got at this point for every sample the program that consists of the prompt, code completion and the tests, the raw uncalibrated probability and the label which indicates if the sample is correct.

Groups

For some calibration methods we discuss later we need features that describe the sample. We call these features groups. A group is a subset of the dataset. These groups can also be intersecting. For example a piece of generated code is written in the language python and has examples in the initial prompt. We could assume that LLMs have a worse structural correctness and therefore calibration with different kind of programs. Maybe they have better calibration on samples which contain examples and a worse calibration on sample with long input prompts. That would be intuitively the case for humans.

The groups are associated with the samples through a matrix. Let N be the number of samples and M be the number of groups. Then the matrix is $N \times M$. This matrix contains zeros and ones which display the affiliation of the sample and the groups. Each row is a sample and each column represents a group. A zero means that the sample does not belong to this group and a one means it does. This matrix is built during the data loading process and will be used in different calibration approaches in the next sections. A sample matrix is displayed in Figure 3.3. The columns G_n stands for the groups and the row X_n stands for the samples. The set G contains all groups. And the function $g(x)$ specifies if the given sample $x \in X$ is in the given group

$$\begin{array}{c} \\ X_1 \\ X_2 \\ \\ X_n \end{array} \begin{array}{ccccc} G_1 & G_2 & \dots & G_m \\ \left[\begin{array}{cccc} 0 & 1 & \dots & 1 \\ 1 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 0 \end{array} \right] \end{array}$$

Figure 3.3: The group matrix has the size $N \times M$, where N is the number of samples and M is the number of groups. The group function $g_m(x)$ assigns the sample x to a group G_m .

In this work we use three different approaches to create groups. The first grouping style we introduce is named „simple“. This grouping style is our the entry point into the group topic. We use simple characteristics to build the groups matrix. The criteria we check for are: the prompt contains the word "example", is the prompt longer than the median prompt, and is the program code longer than the median program code. The idea behind the example group is that it should intuitively be easier to create a program if you have examples that show how the code works. The same goes for the group that checks the prompt length. Here we we assume that the problem becomes more complex the longer the prompt is. For the last group, which checks the length of the generated program, we assume that longer generated code is more likely to be incorrect than shorter code.

Our second grouping style approach is called „complexity“. For this we analyze the generated programs. This is done with command line tool called „scc“ which was created by Ben Boyter [14]. This tool outputs the lines of code, lines of comments and an approximation of cyclomatic complexity per program. The cyclomatic complexity is a metric that measures the independent paths through a program and was introduced by Thomas McCabe [6]. It should be noted that the complexity score we use is only an approximation because its difficult to automate the calculation over multiple different program languages because of their differences in programming paradigms [14]. We choose to create a group for different complexity levels as defined by Thomas McCabe [15]. McCabe [15] introduces a measure of complexity levels which determine risk. The first group contains all samples that have a cyclomatic complexity of 1-10. These samples are considered to have a small risk. The second group

contains all samples that have a complexity of 11-20 and have moderate risk. The third group contains all samples with a complexity of 21-50 and have a high risk. The last group contains all samples that have >50 complexity and have a very high risk. These four groups contain all samples and are therefore self-contained.

The third grouping style places the samples into groups which represent semantic categories of programming problems. These categories are extracted from the task definition that is contained in the prompt. To achieve this, we check the prompt for certain keywords and add them to the group. The first group is build to represent tasks that use mathematical operations and numbers. This assignment takes place through the check for keywords like 'integer', 'float' and a combination of 'number' and 'calculate'. The second group represents all samples that operate with strings or chars. Therefore we search for keywords like 'string', 'char' or 'word'. The last group contains all samples that use higher level data objects like lists. Here we check for keywords like 'list', 'array' or 'tree'.

3.2 Evaluation through a LLM

One straightforward approach comes to mind to estimate our desired probability

$P(y = 1|program)$: We could simply ask another LLM if the generated code is correct. Such auto-evaluation is quite common in question, answering, for example <https://autoevaluator.langchain.com/>.

We also created a method to evaluate the generated code with the help of a LLM. The LLM can either be the same LLM or we can choose another one. This is helpful if want to evaluate the generated code with a more powerful LLM. To get an evaluation of the generated programs, we build a prompt which contains the programming language, the program, which contains prompt, generated code and test cases, and an instruction to evaluate the correctness of the code. The actual prompt templates can be found in the appendix A.1. The prompts and approaches are inspired by the work of Ulmer et al. [9]. Note that the evaluator LLM does not need to be a open source model and it is also not necessary to retrieve the log probabilities. This is possible because we directly use the generated token sequences from the evaluator LLM and do not use the log probabilities.

For the evaluation of the generated code we differentiate between a quantitative and a qualitative approach.

- The quantitative approach tries to directly get a percentage of correctness from the model. Therefore the model is asked to return a percentage of correctness from the range of 0 to 100. This percentage of correctness is then treated as the confidence in the correctness of the sample. We assume that it is hard for a model to rate the program by using a continuous number range.
- The qualitative approach, on the other hand, gives the model the option to choose from a preset of qualitative descriptions. For example „High“ or „Very low“. These qualitative scale can be found in Table 3.1. Based on the selection the model makes the sample gets assigned a specific percentage. These values are defined independently and can also be adjusted after the model made its prediction.

For both methods we use template matching to extract the values from the answer token sequences. For the qualitative approach we search for a number with a percentage symbol and for the qualitative approach we search for the predefined qualitative descriptions. Therefore we use the first occurrence.

The probability outputs of these method can than be used for further calibration methods in the process. It is also possible to not use any calibration method and analyze the pure outputs from the

Description	Probability
Very low	0
Low	0.3
Somewhat low	0.45
Medium	0.5
Somewhat high	0.65
High	0.7
Very high	1

Table 3.1: Qualitative scale mapping which is used to rate the correctness of the generation with the evaluator LLM. If the evaluator LLM returns the description low the sample gets a probability of 30%.

evaluator LLM.

3.3 Calibration foundations

For our calibration approaches we need a probability which reflects the confidence in the generated code. We obtain a first baseline estimate of this confidence from the individual token probabilities. We assume that if the model is „insecure“ about its generation these token score will be low / have a low entropy. With them we calculate the average token log probability per sample. This is done by accumulation of the token log probabilities. This number is then divided by the number of generated tokens. The result is then transformed into actual probabilities by using the exponential function. The formula is displayed in 3.1. We assume that these average probabilities are the confidence of the model in the generated code for further experiments. Lower average token probabilities imply an uncertainty in the model output. Higher confidence scores imply that the model thinks that its output is more reliable. As a results of this process step, we get an probability for each sample that represents the confidence.

$$p^{raw}(x) = \exp\left(\frac{\sum_{k=1}^n \log prob_k}{n}\right) \quad (3.1)$$

These uncalibrated probabilities of our code samples are the baseline for our further work. We will use them to verify if we can achieve a better calibration with the calibration methods we display in the following sections.

3.3.1 Discretizing scores

To review the calibration on our generated data, we need to split the probability range into bins. This is done by splitting the range into equally large bins. They are defined by $[\frac{1}{m}] = \{0, \frac{1}{m}, \frac{2}{m}, \dots, \frac{m-1}{m}, 1\}$. We control the number of bins with the parameter m . To assign this probabilities to a bin we need to know which probability comes into which bin. To achieve this we calculate the distance to each bin and choose the bin which is closest to the probability. The formula to calculate said assigned bin can be seen in 3.2.

$$P'(x) = \operatorname{argmin}(|p^{raw}(x) - b|), \text{ where } b \in [\frac{1}{m}] \text{ and } x \in X \quad (3.2)$$

This mapping is then used to calculate the correctness in each bin b . This is done before and after the calibration method was applied. Through that we can evaluate the calibration on the uncalibrated data, the calibration on the calibrated data and the increase or decrease of metrics that were showed in Chapter 2.

3.3.2 Visualizing calibration result

To compare the approaches we calculate metrics on the uncalibrated data and then after each method was used. Through this we get an idea of how well each method performed under the same circumstances. We can now use we use reliability charts and histograms to plot the changes that each calibration method applies. The reliability charts show each bin and the correctness of the samples in said bin. The histograms show the distribution of the samples in the grid and can therefore track where the distribution mass moved. In Figure 3.4 we can see an example for a reliability chart. On the x-Axis we got the confidence values that we received from the LLM. On the y-Axis we got the probability that a sample is correct for each bin. The blue diagonal line symbolizes a perfect calibration. The bar colors of the reliability diagram display the sample distribution. Darker blue shades mean more samples are in this bin and light blue or white displays less samples. Furthermore the numbers about the bars show the actual count of samples in this bin.

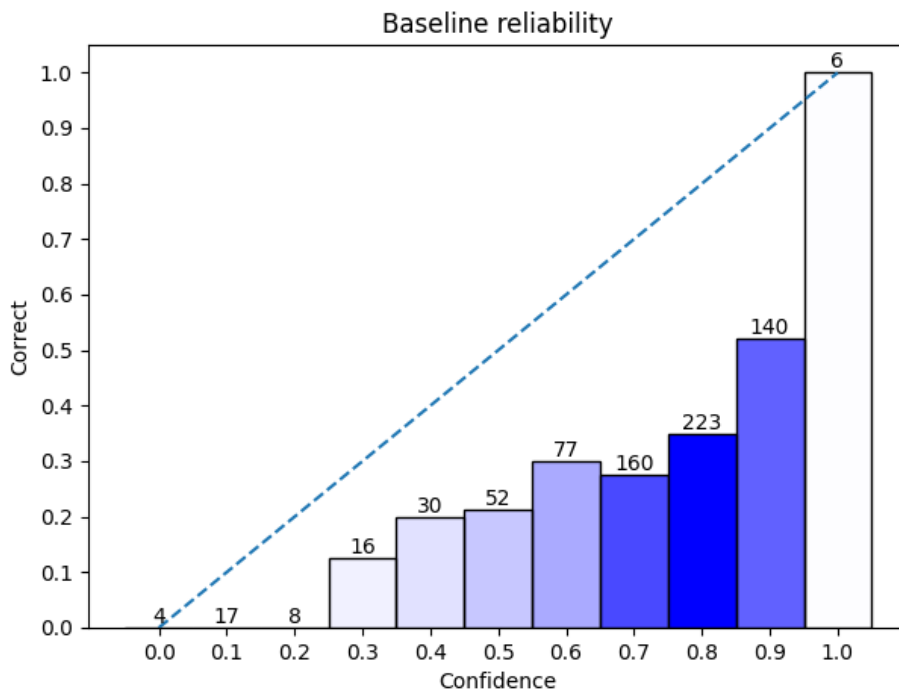


Figure 3.4: Reliability chart to display calibration. The x-axis displays the confidence of the LLM and the y-axis shows the correctness in each bin. Above each bar are the number of samples in the bin. The dotted line indicates perfect calibration. The color gradient shows where the most samples are located. Darker blue means more samples.

In Figure 3.4 we can see the uncalibrated data distribution of a dataset. As we can see the probabilities are unevenly distributed and are not close to the diagonal line which would be the perfect calibration of the underlying data. Perfectly calibrated data would mean that in bin 0.5 are always 50% correct and 50% incorrect samples. The bin with a confidence 0.8 contains 223 samples. It is the bin with the most samples and there for has the darkest shade of blue. The raw uncalibrated probabilities express a high confidence (80%). But we can see that samples in this bin are have a correctness of around 35%. This is an example of poor calibration. For a good calibration the bar should be at 80%.

3.4 Calibration

A calibration method is something that gets the raw uncalibrated probabilities and turns them into better calibrated probabilities. We hope that these better calibrated probabilities provide a better estimation of the correctness of a generated code sample. Some of these methods do not only take the raw uncalibrated probabilities, but also use groups. These groups are used to obtain a better calibration on the subset of samples that are contained in this groups. We hope to achieve better calibration by choosing a good combination of groups.

3.4.1 Histogram binning

The first calibration approach is called histogram binning (HB). This approach is the simplest of those studied in this thesis and tries to adjust all probabilities in a bin at once. To achieve the calibration improvement, we calculate the difference between the correctness and the average confidence in the bin for all defined bins. These so called deltas in formula 3.3 are learned under supervision on a trainings set. Because we discretized the values in each bin the average confidence is always the value of the bin. For example in the bin 0.5 the average confidence is 0.5. The correctness is calculated by using the label of the samples. With the these two values we calculate the difference (delta) between optimal calibration and actual calibration. This is done for every bin.

$$\text{delta}_b = E[P(y = 1|x) - P'(x)], \text{ where } b \in [\frac{1}{m}] \text{ and } x \in S_b \quad (3.3)$$

The calculated deltas are then used to update all discretized probabilities in each bin. These updates are made to every defined bin. This is done by adding the corresponding delta with the discretized probabilities in the selected bin. The result of this are new adjusted probabilities for every sample. This process shifts the probabilities to a better calibrated version. But it neglect the fact that different bins and the contained samples are not always disjoint and do maybe depend on each other [3].

3.4.2 Linear regression

The second method we use to calibrate our raw uncalibrated probabilities is linear regression (LR). In this approach the groups appear. The groups are used as input features for the linear regression. In formula 3.4 these groups are displayed as x_i and can take the value one or zero. As output we want the difference between the label and the actual probability which is displayed as y . The label defines if a sample is correct or not and therefore can take the values zero or one. Which means incorrect sample will be shifted into the direction of zero because they become a negative sign and the correct sample will be shifted more to the direction of one because of the positive sign.

$$\text{delta} = b + w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n \quad (3.4)$$

To realize this the linear regression learns the weights w_n for each group and the bias b . In this approach we use the actual probabilities and not the discretized values. Further more the bias b of the linear regression is used to adjust samples that have no groups associated with. These weights and the bias are learned on a training set.

With the learned weights $w_1, \dots, w_n$ and bias b we predict the adjustment for a given group feature input $x_1, \dots, x_n$. The predicted y value is then added to the raw uncalibrated probability of the sample. *new probability* = $y + \text{probability}$. Through that we get a better calibrated probability for the sample.

We also conducted a control experiment in which the actual labels are learned. The results will be shown in Chapter 4.

3.4.3 Iterative group histogram binning

Iterative group histogram binning (IGHB) is a approach which combines the idea of groups with the concept of bins. Instead of calculating deltas for each bin, like we did it in the histogram binning 3.4.1 approach, we calculate these deltas for every bin-group combination. That is done by calculating the difference between correctness and the average confidence for every bin and the groups in the given bin as we see in Formula 3.5. Through this we create a matrix of delta values. Where M is the number of bins and G is the number of groups. Then we get matrix with the shape of $M \times G$.

$$\text{delta}_{b,g} = E[P(y = 1|x) - P'(x)], \text{ where } b \in [\frac{1}{m}], g \in G \text{ and } x \in S_{b,g} \quad (3.5)$$

The problem that arises is that we now have multiple delta values for a sample. That is because a sample can be in multiple groups, but only can be in one bin. So instead of applying the deltas to every sample in our dataset, like we did with histogram binning, we select the combination of bin and group which has the most impact on the calibration. To calculate this we square our deltas 3.5 and multiply them with the probability that a sample is in the bin-group combination. For the formula see 3.6 By doing this we get the bin and group which has the highest impact on the calibration of the data.

$$(b, g) = \text{argmax}(\text{delta}_{b,g}^2 * P(S_{b,g}), \text{ where } b \in [\frac{1}{m}] \text{ and } g \in G \quad (3.6)$$

The adjustments of the sample probabilities are made in a loop. This loop stops if the maximal error in the data is smaller then a predefined value α . This α value also defines the grid size in this approach. The α value of 0.1 results in a grid of 10 bins. So the grid is defined by calculating $\frac{1}{\alpha}$ and using this value as the bin width. The max error on the other hand is recalculated on each iteration of the loop. It is defined as the result of multiplying the GASCE with the probability of which a sample is in a given group as we can see in formula 3.7.

$$\text{max error} = \text{argmax}(\text{GASCE} * P(g)), \text{ where } g \in G \quad (3.7)$$

In each iteration we adjust the probabilities on all subsets of our data. For evaluation purposes we track the changes which each iteration made. After each iteration we fit the calibrator with the adjusted probabilities. That means we calculate each deltas again and check if the error is still greater than the α value. Is that not the case we stop the adjustment and the calibration process is finished.

Through the specific adjustment we make for the bin-group combinations there is a high risk of

overfitting on the data. That is caused by the small amount of data that is adjusted in each iteration.

To apply the learned changes onto an new sample we need to keep track of every adjustment that is made per iteration. Therefore we need to save the bin-group combination that was adjusted and the delta which was applied. It is important to mention that these changes need to be applied in the exact same order as they were saved. That is because through the changes of time samples are moved into other bins and could be affected by the next change in another way.

3.4.4 Iterative group linear binning

Iterative group linear binning (IGLB) improves the previous IGHB approach. Therefore three adjustments are made. First we not only apply the corrections for exactly one bin-group combination but to a cumulative bins instead of individual ones. This set is defined by the parameter τ . A τ of zero means we adjust all values that are smaller than or equal the selected bin. Analog a τ of one means we adjust all values that are greater than or equal the selected bin. The parameter τ specifies this cumulative distribution function. The update of a larger number of samples is assumed to help to limit the risk of overfitting.

Secondly we apply so called linear scaling within each bin. This scaling is defined by the formula 3.8:

$$p^{IGLB}(x) = \text{expit}(\alpha + \beta * \text{logit}(P(x))) \quad (3.8)$$

X are the probabilities of the samples and X' are the new adjusted probabilities. To get an optimal set of alpha and beta values we minimize the MSE on Formula 3.8. This is done by using the Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm. As initial start parameter we use 0 for alpha and 1 for beta. By applying the formula we calculate the alpha and beta values for all tau-bin-group combinations. In the prediction phase we select the set which is needed for the adjustment.

Algorithm 1: Iterative group linear binning

Data: $X_{train}, X_{test}, X_{val}$
 $Y_{train}, Y_{test}, Y_{val}$
 $G_{train}, G_{test}, G_{val}$

Result: $\hat{P}(X)$

$\epsilon = 0.01$;

/* adjustable parameter */

while True **do**

$mse_t \leftarrow \text{MSE}(f(X_{val}), Y_{val})$;

$P'(X_{train}) = d(X_{train})$;

$P(S_{\tau,b,g})$;

 TODO;

$mse_{t+1} \leftarrow \text{MSE}(\hat{X}_{val}, Y_{val})$;

if $mse_{t+1} \geq mse_t$ **then**

 | break

end

end

We choose the subset we correct by calculating the probability that a sample is in a given subset with parameter tau, group and bin. We also do this analog with the delta values. This probability is then multiplied by the deltas squared. We get the tau-bin-group combination with the largest potential to

improve calibration. This calculation is formalized in the formulas 3.9 and 3.10.

$$\text{delta}_{\otimes, b, g} = E[P(y = 1|x) - P'(x)], \text{ where } b \in [\frac{1}{m}], g \in G, \otimes \in \{\leq, \geq\} \text{ and } x \in S_{\otimes, b, g} \quad (3.9)$$

$$(\otimes, b, g) = \text{argmax}(\text{delta}_{\otimes, b, g}^2 * P(S_{\otimes, b, g})), \text{ where } b \in [\frac{1}{m}], g \in G \text{ and } \otimes \in \{\leq, \geq\} \quad (3.10)$$

The third improvement specifies conditions that lead to an early stopping of the algorithm. The early stopping is to prevent overfitting. The first early stopping criterion is the parameter ϵ . If ϵ is greater then the probability $P(S_{\otimes, b, g})$ of the chosen subset the algorithm will terminate. The second criterion checks if the MSE is still decreasing on the validation data set. If it stops decreasing or stays the same the algorithm will come to an halt.

To apply the changes onto a new sample, that has not been learned, we need to apply all changes that where made during the iteration onto the sample. The changes need to be in order the where done while training.

3.4.5 Regressor

In the last approach we choose to use different regressors. With these regressors we predict the correction terms which are used to update the probabilities like in the approach 3.4.2. But with the modification that we use different input features for the regressor.

The new input features consists of the sample groups, the assigned sample probability and a histogram. The groups did not change and are used like we described them in Section 3.1. For the probability we used the average token probability. The histogram was created by gathering the individual token log probabilities. These were transformed into normal probabilities and summarized using a histogram. The histogram bins were divided into ten bins. A sample histogram can be seen in Figure 3.5. We assume that the distribution of token scores can correlate with the correctness of a sample.

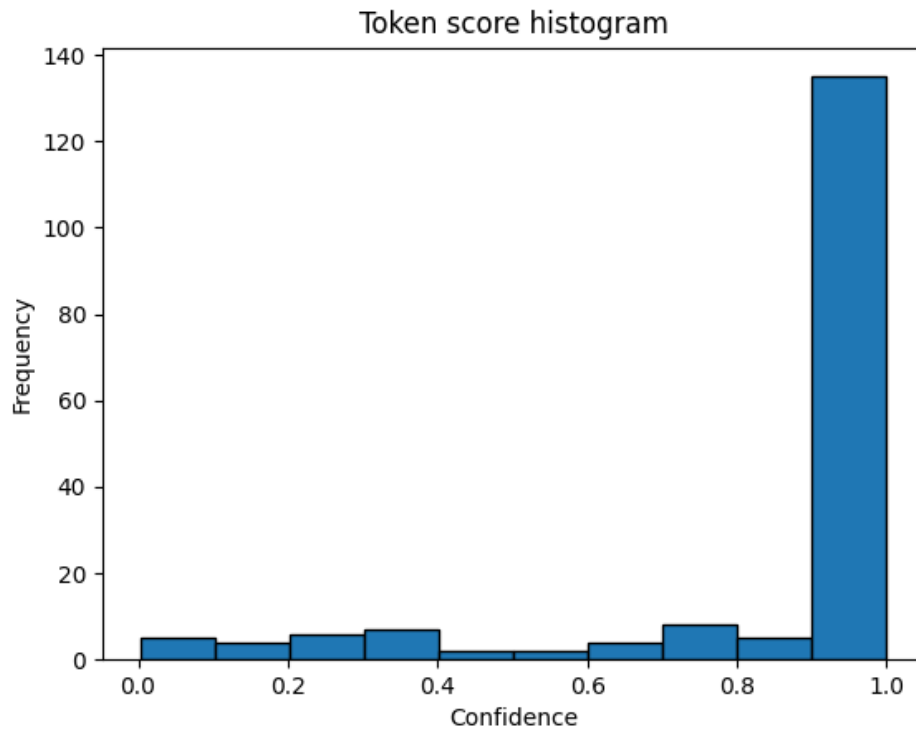


Figure 3.5: Histogram that displays the distribution of the token confidence in the humaneval task 19. The confidence is displayed on the x-axis and the number tokens in each bin are on the y-axis. As we see the model is mostly overconfident in its token selection.

This input features were used to train the regressor. As target we predict the deltas to adjust the probabilities for better calibration. As regressor we use linear regression, support vector regression (SVR) and XGBoost.

The linear regression is used like described in Section 3.4.2, but with the modified input features.

For the SVR we use the radial basis function and for XGBoost we used the default settings to fit our data.

All three regressors get the same input features and same target value. We input our features and learn the deltas for calibration. For the prediction part we apply the outputs of the regressor to the sample probability. We assume that the regressors learn deltas that improve the calibration of our data. At this point no bin assignments are made for the calibration process.

Chapter 4

Evaluation

Our goal is to evaluate if the calibration approaches show an improvement in calibration. Furthermore we show which approach provides the best results. We display the datasets and benchmarks we used. Additionally we show how the code was evaluated. After this we will display the used models and give an insight of the parameters that were used for the generation process and the metrics which were used for the evaluation of the samples. We also conducted experiments which look into some aspects of our calibration approaches. For this we answer a specific research question for each experiment.

4.1 Data

As data for our experiments we mainly use the humaneval dataset [2]. This data consists of 164 programming tasks which are written in python. We already displayed one of these prompts in Listing 3.1. The tasks are built to assess language comprehension, reasoning, algorithms and mathematics [2].

These 164 samples are slightly too few for our calibration approaches; we choose to use the MultiPL-E benchmark [1]. The benchmark makes it possible to translate the humaneval tasks into 22 other programming languages. Through this we get a sample count of approximately 3600. They use a compiler to translate the unit tests, types, doctests and the python terminology. So the evaluation for correctness works like for the python tasks with unit tests. The samples are evaluated in a docker container. That is because executing generated code can be dangerous. The generations are cut off at a certain stop word. For python this is for example the word „\ndef“ which would mark the start of a new high level function.

The generation works over a command line tool where we specify several parameters. These parameters are the model, the dataset (either humaneval or MBPP), the programming language, temperature, batch size, completion limit and our output directory. In default these benchmarks do not return the log probabilities. To return them we made some changes to the existing MultiPL-E scripts. We added the logprob parameter in the automodel_vllm script and adjusted the results files to contain the token ids, log probabilities and the cumulative log probabilities. We generate only one generation for each sample and use batch processing to generate multiple programs at once.

For this work we investigate the generalization abilities of the different calibration approaches. Therefore we train the calibration approaches on 60% of our data. The other 40% are used for validation

(20%) and testing (20%) of the learned parameters. We decided to use cross validation only in certain cases because the data is sufficiently large enough. The validation subset is necessary for the IGLB calibration approach and is used for early stopping.

The output of each method is stored in a folder structure. This folder structure is created while loading the data for the calibration approach. It changes dynamically when the approach is called with another parameter set over the command line. The following is an example of how the structure looks: „runs/humaneval-all-keep-Qwen2.5_Coder_7B-Instruct-1.0-comp-1/lr/linear/avg_logprob/categories/split“. The runs folder contains all results that are generated. The second folder is the run which describes the dataset and for code generation also the model with which the samples were generated. After this the calibration approach is specified in this case its „lr“ and under this folder we find the different binning types. For this work we only considered linear binning so its the only possible option. The next option specifies which method is used to generate the probability which is associated with the sample. The options are the average token probability (avg_logprob), the quantitative or the qualitative probabilities which were generated by the evaluation LLM. After this we got a folder which specifies the grouping style which were explained in the section 3.1. The last folder contains either the results for the partitioned data or for the run with all data and no split into train, test and validation.

In the deepest folder we find charts which display the calibration before and after the method was used and a table which contains the calculated metrics.

4.2 Models and parameters

The model is loaded through vllm and is then used to create the code generations. As model we used Qwen2.5-Coder-7B-Instruct. At the time of this work this is the best performing 7B-Model on the BigCodeBench Leaderboard [13]. BigCodeBench is a benchmark that involves solving tasks with code. It focuses on different functions and complicated instructions.

This model was loaded from the huggingface hub. For the temperature we choose 1 and kept the default value for top-p as 0.95.

4.3 Results

In this section we first take a look into the calibration of our baseline. Then we show the results of the different calibration approaches. Therefore we look into the improvement of calibration compared to our baseline. The baseline uses the average token log probabilities per sample. We did not use any splitting for the baseline. This is done to get an overview of our data. To create our grid we use the parameter $m = 10$. This creates a grid of eleven bins. That means all our probabilities get discretized to values in 0, 0.1, 0.2, ..., 0.9, 1.0. This grid will be used to display the calibration results for all used approaches. This enables us to compare the approaches later on.

humaneval-all-keep-Qwen2.5_Coder_7B-Instruct-1.0-comp-1 # Calibration Bar Chart

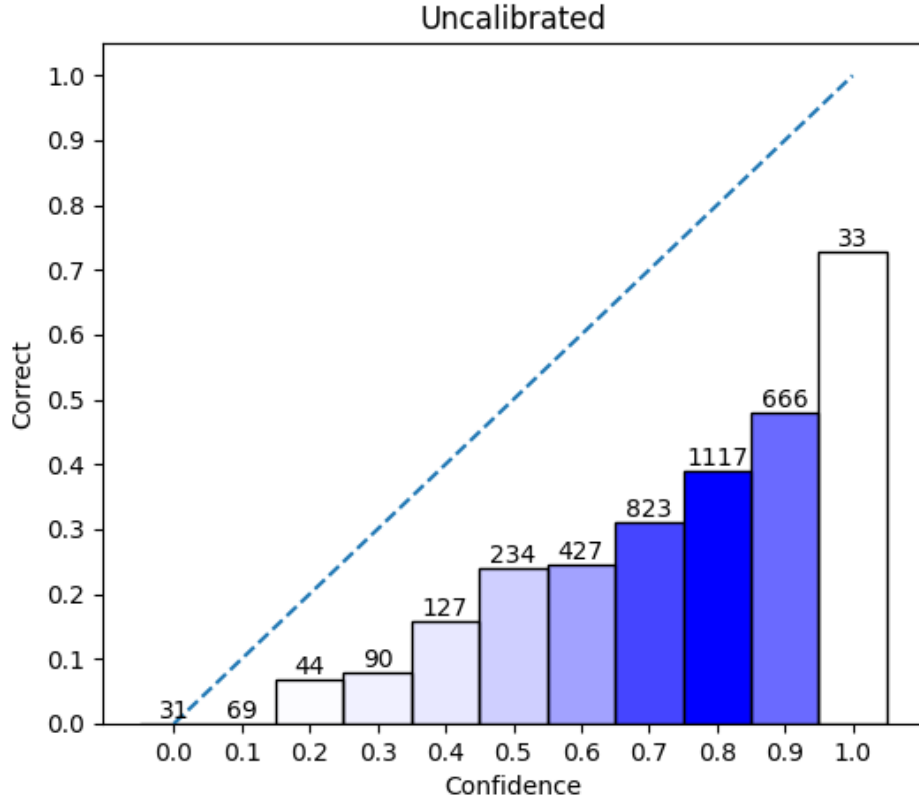


Figure 4.1: Reliability chart that displays the baseline calibration. On the x-axis we got the confidence of the model in the generated code and on the y-axis we got the correctness in each bin. We can see that the majority of the samples are located around the confidence of 80%.

In Figure 4.1 we see the baseline for our following experiments. As we can see the most samples are around the bin with a confidence of 80%. This means that the LLM overestimates the correctness of the code generations. Only the bin with a confidence of 0% has perfect calibration. Every other bin is not well calibrated. For example the 50% bin has only a correctness of around 25%. Which means these samples should be in the bin with a confidence of 30%, because the discretization from Formula 3.2.

ECE ↓	ASCE ↓	MSE ↓	MSE _{ref}	Skill Score ↑
0.37	0.1405	0.3489	0.22	-0.59

Table 4.1: Baseline metrics of the uncalibrated average token probabilities. The scores are for the MulitPL-E Benchmark with the humaneval dataset and the Qwen2.5-Coder-7B-Instruct model.

Table 4.1 shows the metrics for our baseline. We can see that the ECE, ASCE and MSE scores are quiet high. The lower these scores are, the better is the calibration. A value of zero for the ECE and ASCE would mean that the model is perfectly calibrated. The MSE ref is a measure of a naive estimator.

This estimator is defined by $MSE_{ref} = p_c * (1 - p_c)$. p_c is the probability of correctness over all our samples. The value of 0.22 means that roughly $\frac{1}{3}$ of our samples are correct. The MSE_{ref} will be used to compare our approaches and is part of the skill score. The skill score measures if the MSE is better then the MSE_{ref} score. If its positive this indicates an improvement of calibration if it is negative it is a indicator for poor calibration. So we can see that our baseline is not well calibrated an therefore needs adjustments.

Group	GASCE ↓
> Median Prompt Len	0.057
not > Median Prompt Len	0.0848
examples	0.0359
not examples	0.1061
> median code	0.0848
not > median code	0.0579

Table 4.2: GASCE of the baseline with uncalibrated average token probabilities. The scores are for the MulitPL-E Benchmark with the humaneval dataset and the Qwen2.5-Coder-7B-Instruct model.

Furthermore we got the values for the GASCE in the Table 4.2. The GASCE makes it possible to measure the calibration error for each group independently. For our baseline we only used the „simple“grouping style. In the later experiments we show the effect of other groups. The Table 4.2 shows each group with there name or descriptive feature and the corresponding GASCE value. We can see that the greatest miss calibration is in the group „**not** examples“. And the best calibrated group is the „examples“group. This means that samples with an example are better calibrated out of the box.

4.3.1 Histogram binning

To recapitulate the histogram binning method used deltas which correct every sample in a bin. This deltas represent the difference between the correctness and the confidence in the given bin. We used the same parameter $m = 10$ to create the grid like in our baseline. This will be done for all the following approaches except the iterative group histogram binning approach. The deltas for each bin where learned on the training subset and applied to the samples in the test subset. We do not mention groups because they do not play a role in this calibration approach.

In Figure 4.2 the results of application of the approach are shown. Left column of the Figure 4.2 represents the test subset before calibration and the right column represents the data after the histogram binning was applied. This kind of diagram will be used for every approach we show in this results section. We see that the distribution of our samples shifted from high confidence values to much lower confidence value. For example the bin 80% which contains the most samples with 233 shifted to the confidence value of 40%. Furthermore the data is better calibrated and lines up better with the dotted line which represents perfect calibration. Nevertheless it did not worked for all bins as we can see in bin with 100% confidence. It was perfectly calibrated before the histogram binning was used. So the approach learned a delta value that does not generalizes well. This could be due the small amount of samples in this bin.

humaneval-all-keep-Qwen2.5_Coder_7B-Instruct-1.0-comp-1 # Calibration Charts

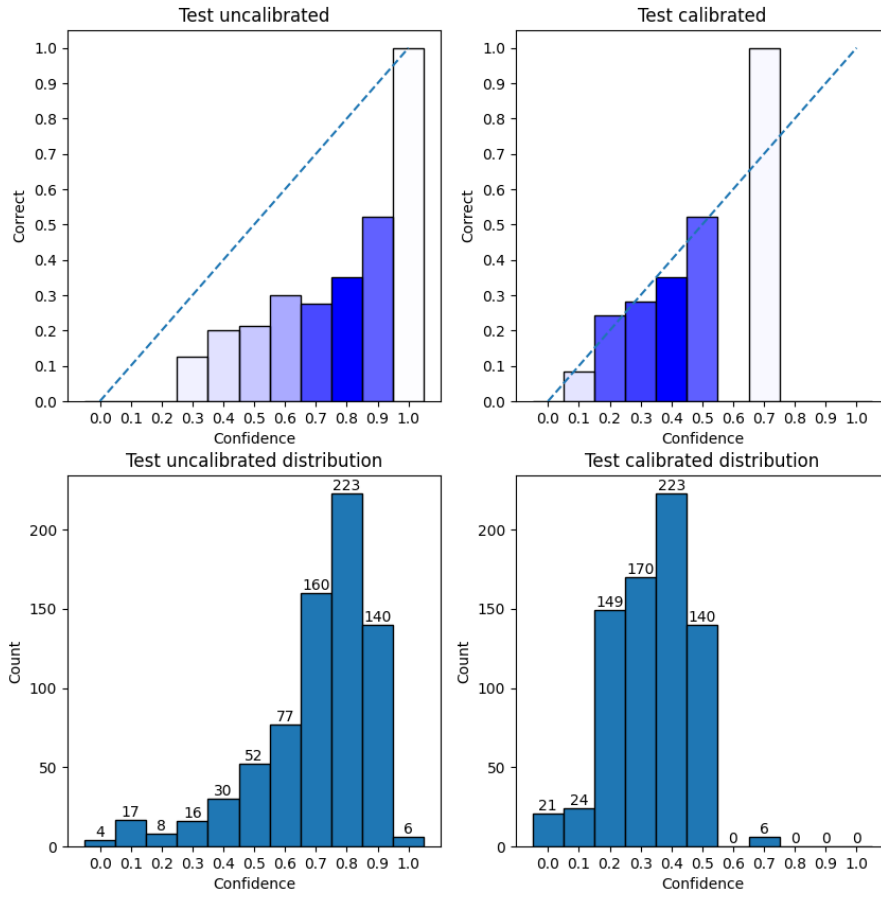


Figure 4.2: Displays the calibration before (left side) and after (right side) the histogram binning approach was used. On the x-axis we got the confidence of the model in the generated code. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution our samples are displayed.

That the approach improved the calibration can also be seen in the metrics we tracked. They are shown in Table 4.3. Both the ECE and the ASCE improved to a value close to zero. This assures us that the results are better calibrated then before hand. Furthermore the MSE decreased by -0.14. Which is not as much as we expected. But Nevertheless the skill score improved by 0.66 and is now positive which indicates a better calibration and an improvement over the naive estimator MSE_{ref} .

	ECE ↓	ASCE ↓	MSE ↓	MSE_{ref}	Skill Score ↑
Uncalibrated	0.37	0.1469	0.35	0.22	-0.59
Calibrated	0.03	0.002	0.2056	0.22	0.07
Difference	-0.34	-0.1449	-0.1444	0	0.66

Table 4.3: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

To sum up the learned deltas in the histogram binning approach generalized somewhat good on the test data. It also achieved a better calibration compared to our baseline with the average token probabilities. Although it does not achieve perfect calibration and still has weakness for bins with low sample count.

4.3.2 Linear regression

Our next method is the linear regression. The goal with this approach is to learn the weights w_n for the defined groups G and a bias b to predict the delta. This delta is then used to correct the raw uncalibrated probability. Note that the corrections are made on the raw uncalibrated probabilities and not the discretized values. This is the first approach which uses the groups as a feature to determine the correction needed. Therefore we take a look into the GASCE and show how the approach improved the group calibration.

The Figure 4.3 shows the visualized change of calibration. Again the left side shows the uncalibrated test data and the right side shows the data after the linear regression was used. As we see the sample distribution shifted to the left. Generally the calibration of each bin improved but we got some bins where the approach did not generalized well. At least for the view of all samples. In the results for the comparison we also take a look into the calibration for each group individually. Generally we can say that the calibration was improved through this approach but also did not achieve perfect calibration.

humaneval-all-keep-Qwen2.5_Coder_7B-Instruct-1.0-comp-1 # Calibration Charts

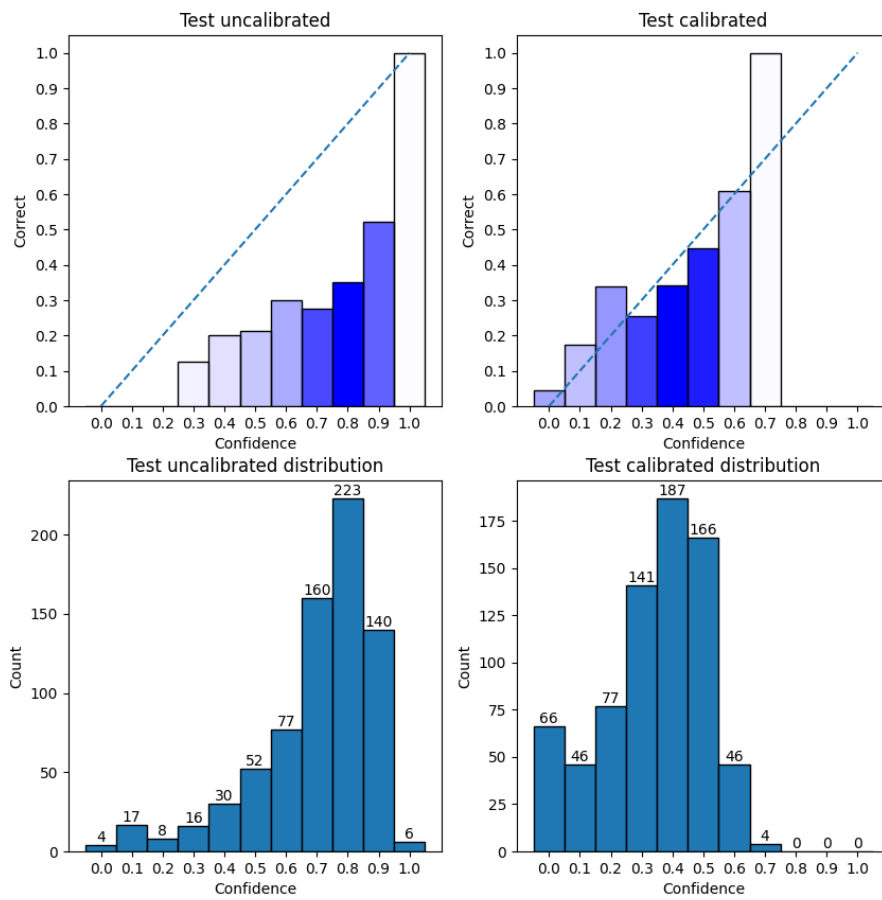


Figure 4.3: Displays the calibration before (left side) and after (right side) the linear regression approach was used. On the x-axis we got the confidence of the model in the generated code. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution our samples are displayed.

If we look into the metrics in Table 4.4 we can see that despite of the visualization the calibration improved quite much on the test dataset. The calibration errors ECE and ASCE improved significantly and also the MSE decreased by -0.14. Also we got a positive skill score that indicates good calibration. Furthermore we can see the GASCE for every group in the Table 4.5. Every group achieved a calibration near zero and only has an error in the three-digit decimal range. This means our groups are well calibrated.

	ECE ↓	ASCE ↓	MSE ↓	MSE _{ref}	Skill Score ↑
Uncalibrated	0.37	0.1469	0.35	0.22	-0.59
Calibrated	0.06	0.0049	0.2062	0.22	0.06
Difference	-0.31	-0.142	-0.1438	0	0.65

Table 4.4: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	GASCE ↓	
	Before	After
> Median Prompt Len	0.0875	0.0025
not > Median Prompt Len	0.0623	0.0044
examples	0.0359	0.0011
not examples	0.1127	0.0043
> median code	0.088	0.0021
not > median code	0.0618	0.0044

Table 4.5: The table shows the GASCE for each group we used. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

To conclude the linear regression approach achieved a good calibration on the overall test dataset. With problems to adjust some samples. Reasons for this could be that the groups are not well chosen. However this opposed by the fact that the GASCE, as we can see in Table 4.5, improved on every group.

4.3.3 Iterative group histogram binning

The iterative group histogram binning used bin-group combinations to make adjustments to the raw uncalibrated probabilities. To use this approach we needed to specify the parameter α which determines the bin width and our stopping condition. For this experiment we chose the value $\alpha = 0.01$. This means we got 101 bins and the algorithm comes to a halt when the max error is smaller then 0.01.

We can see in Figure 4.4 that the distribution of our samples shifted again to the left side. This means the actual probabilities were too high and through the calibration process we got lower confidence values in the correctness of our samples. We can see that the calibration improved because the most samples are around the bins 30%, 40% and 50% which are reasonably good calibrated. The other bins are, besides the 20%, are not well calibrated. A reason for this could be that the algorithm did not targeted the samples in this bins, because the error is too low through the low sample count.

humaneval-all-keep-Qwen2.5_Coder_7B-Instruct-1.0-comp-1 # Calibration Charts

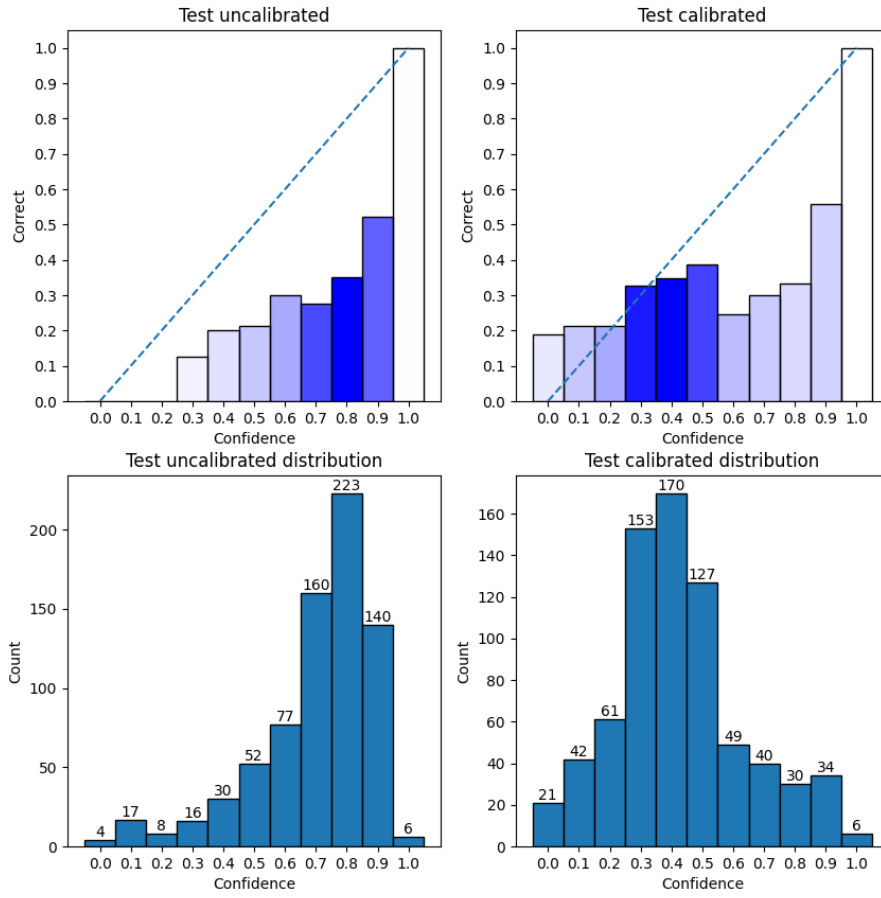


Figure 4.4: Displays the calibration before (left side) and after (right side) the iterative group histogram binning approach was used. On the x-axis we got the confidence of the model in the generated code. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution our samples are displayed.

For further evaluation of the iterative group histogram we take a look into the metrics in the Table 4.6 and Table 4.7. On the ECE we can see an improvement of 0.24. Which means that our bins are have a lower miss calibration then before. The ASCE and MSE also improved. We can see that the skill score does not reach a positive value. This indicates that the calibration is worse then the naive estimator baseline M_{ref} . Furthermore we see in Table 4.7 that the GASCE decreased on all groups. Especially the „not examples“ groups achieved a way better calibration.

	ECE ↓	ASCE ↓	MSE ↓	MSE _{ref}	Skill Score ↑
Uncalibrated	0.37	0.1469	0.35	0.22	-0.59
Calibrated	0.13	0.0363	0.2482	0.22	-0.13
Difference	-0.24	-0.1106	-0.1018	0	0.46

Table 4.6: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	GASCE ↓	
	Before	After
> Median Prompt Len	0.0875	0.0202
not > Median Prompt Len	0.0623	0.0188
examples	0.0359	0.0236
not examples	0.1127	0.0161
> median code	0.088	0.0182
not > median code	0.0618	0.0192

Table 4.7: The table shows the GASCE for each group we used. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

We conclude that the approach slightly improves calibration, but has the tendency to overfit on the trainings data. Because of this tendency we decided to test the approach with cross-validation with a 5-Fold. The results showed that the generalization is poor on all folds.

4.3.4 Iterative linear histogram binning

Our last approach is the iterative linear histogram binning. This is a improvement of the previous iterative group histogram binning approach. The main improvements are the adjustment on a group of bins, the linear scaling in said groups and the early stopping with the MSE on the validation data set and the parameter ϵ . For ϵ we choose the value 0.01. Note that the bin width is again determined by the value m and not by the α like in the iterative group histogram binning approach.

The visualized results of the approach can be seen in Figure 4.5. Before the approach was used the most samples have a confidence in the range of 70%-90% and the only calibrated bin is the 100% which contains only five samples. After the approach was used the most samples have a confidence between 30%-50%. The confidence greatly reduced but is now more in line with the correctness value of these bins. The bins with less samples 60%-90% are not well calibrated after the calibration process.

humaneval-all-keep-Qwen2.5_Coder_7B-Instruct-1.0-comp-1 # Calibration Charts

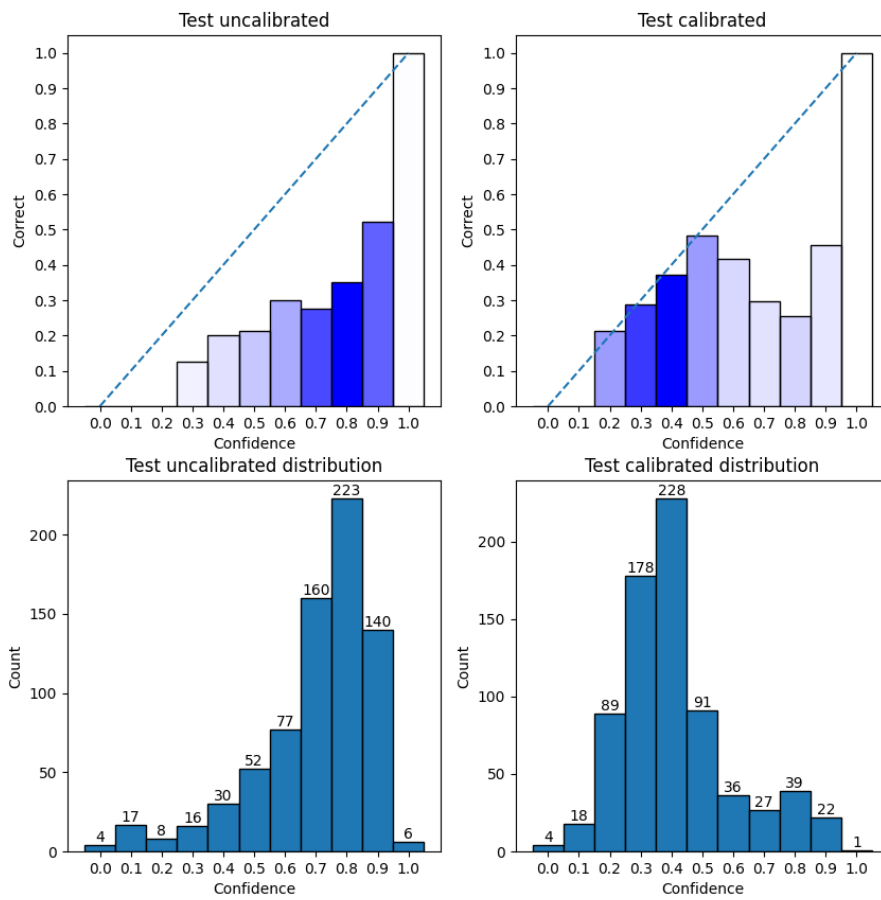


Figure 4.5: Displays the calibration before (left side) and after (right side) the iterative group linear binning approach was used. On the x-axis we got the confidence of the model in the generated code. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution our samples are displayed.

The metrics in Table 4.8 show that the calibration has improved. The ECE decreased by 0.29, the ASCE by 0.117 and the MSE by 0.11. The skill score measure shows also an improvement against our raw uncalibrated probability baseline, but it does not achieve a positive value. The GASCE, which can be seen in 4.9 improved on all groups. However the best improvement achieved the „not examples“ group, which has a GASCE of nearly zero.

	ECE ↓	ASCE ↓	MSE ↓	MSE _{ref}	Skill Score ↑
Uncalibrated	0.37	0.1469	0.35	0.22	-0.59
Calibrated	0.08	0.0299	0.2409	0.22	-0.1
Difference	-0.29	-0.117	-0.1091	0	0.49

Table 4.8: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	GASCE ↓	
	Before	After
> Median Prompt Len	0.0875	0.0201
not > Median Prompt Len	0.0623	0.0123
examples	0.0359	0.0359
not examples	0.1127	0.0007
> median code	0.088	0.0201
not > median code	0.0618	0.0122

Table 4.9: The table shows the GASCE for each group we used. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

To summarize the iterative group linear binning approach achieved a better calibration against our baseline. Nevertheless it should be mentioned that the strong improvement of only one group is an indicator that the algorithm came quickly to a halt. Reasons for that could be the stagnating MSE on the validation set or an improper ϵ value.

4.3.5 Comparison

To summarize we now compare the approaches and check which one achieved the best calibration results. Therefore we first compare them visually with each of the reliability diagrams as above. This can be seen in Figure 4.6. Then we look into the calibration which is achieved inside each group and check the metrics for each method.

Figure 4.6 shows the reliability diagrams for each method. Confidence on the x-axis and correctness on the y-axis. The chart in the first row displays the baseline calibration on the test subset. The second row contains the histogram binning and linear regression charts. The last row the iterative group histogram binning and iterative group linear binning results.

We can see that visually the histogram binning and linear regression approaches show the best calibration results. The iterative group histogram binning approach on the other hand looks slightly better calibrated, but worse then the other both approaches. The improved version, the iterative group linear binning, show again better results the the iterative group histogram binning approach but worse then histogram binning and linear regression.

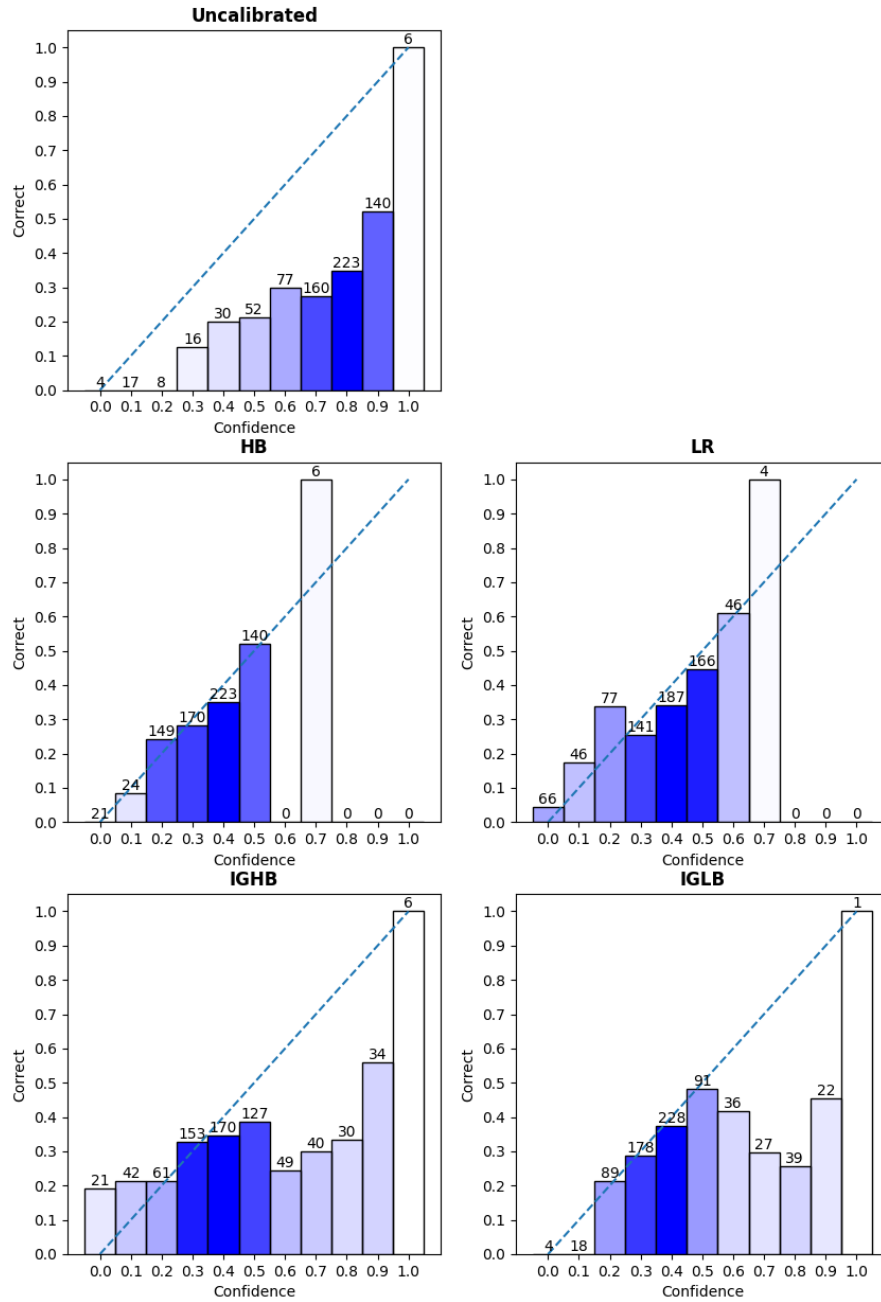


Figure 4.6: Displays the calibration before and after each approach was used. On the x-axis we got the confidence of the model in the generated code. The y-axis displays the correctness in each bin. Over each bar the samples count is displayed. The first reliability chart shows the uncalibrated data. The other charts stand for each approach we used.

What we see visually also reflects in the metrics which are displayed in Table 4.10. The uncalibrated row stands for the baseline with average token probabilities. All other rows are the corresponding

approaches we already described in previous sections. We can see that the simple histogram binning approach works the best on the metrics with lowest ECE, ASCE, MSE and highest skill score. Not far off comes the linear regression which was the first approach that used groups to achieve a better calibration. The worst performance has the iterative group histogram binning with highest ECE, ASCE, MSE scores and lowest skill score. That is the case of the poor generalization ability and overfitting on the training dataset. The improvements that were introduced with the iterative group linear binning have enhanced the metrics as we can see in the last row. Nevertheless the improvements were not enough to beat the histogram binning or linear regression approach.

Method	ECE ↓	ASCE ↓	MSE ↓	MSE _{ref}	Skill Score ↑
Uncalibrated	0.37	0.1469	0.35	0.22	-0.59
HB	0.03	0.002	0.2056	0.22	0.07
LR	0.06	0.0049	0.2062	0.22	0.06
IGHB	0.13	0.0363	0.2482	0.22	-0.13
IGLB	0.08	0.0299	0.2409	0.22	-0.1

Table 4.10: Shows the metrics for each approach we used for comparison. The bold scores indicate the best value for said metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

The next Figure 4.7 is structured the same as Figure 4.6. The charts show the calibration per group. Each circle represents one group. The circle diameter indicates the number of samples in the group. The assignment is as follows:

- > Median Prompt Len ●
- **not** > Median Prompt Len ●
- examples ●
- **not** examples ●
- > median code ●
- **not** > median code ●

On our baseline every group has an approximated confidence of 70%. Every groups is far off from being well calibrated. In the diagram for the histogram binning approach we can see that the groups were just shifted to a lower confidence value. This makes sense because we did not use the groups in this approach. The linear regression on the other hand achieves a solid calibration for nearly every group. The yellow and blue group are slightly off the perfect dot calibration line. Furthermore the confidence shifted from 70% to a range of 25%-40%. Iterative group histogram binning betters the calibration on nearly every group except the green group which has less samples than the other ones. It is expected that the group with low sample count has a worse calibration, because the error in this group will be lower. That is because the probability of each group plays a role in calculating said error. The improved iterative group linear binning approach achieved better calibration than its predecessor, but also does not succeed in correcting the green group.

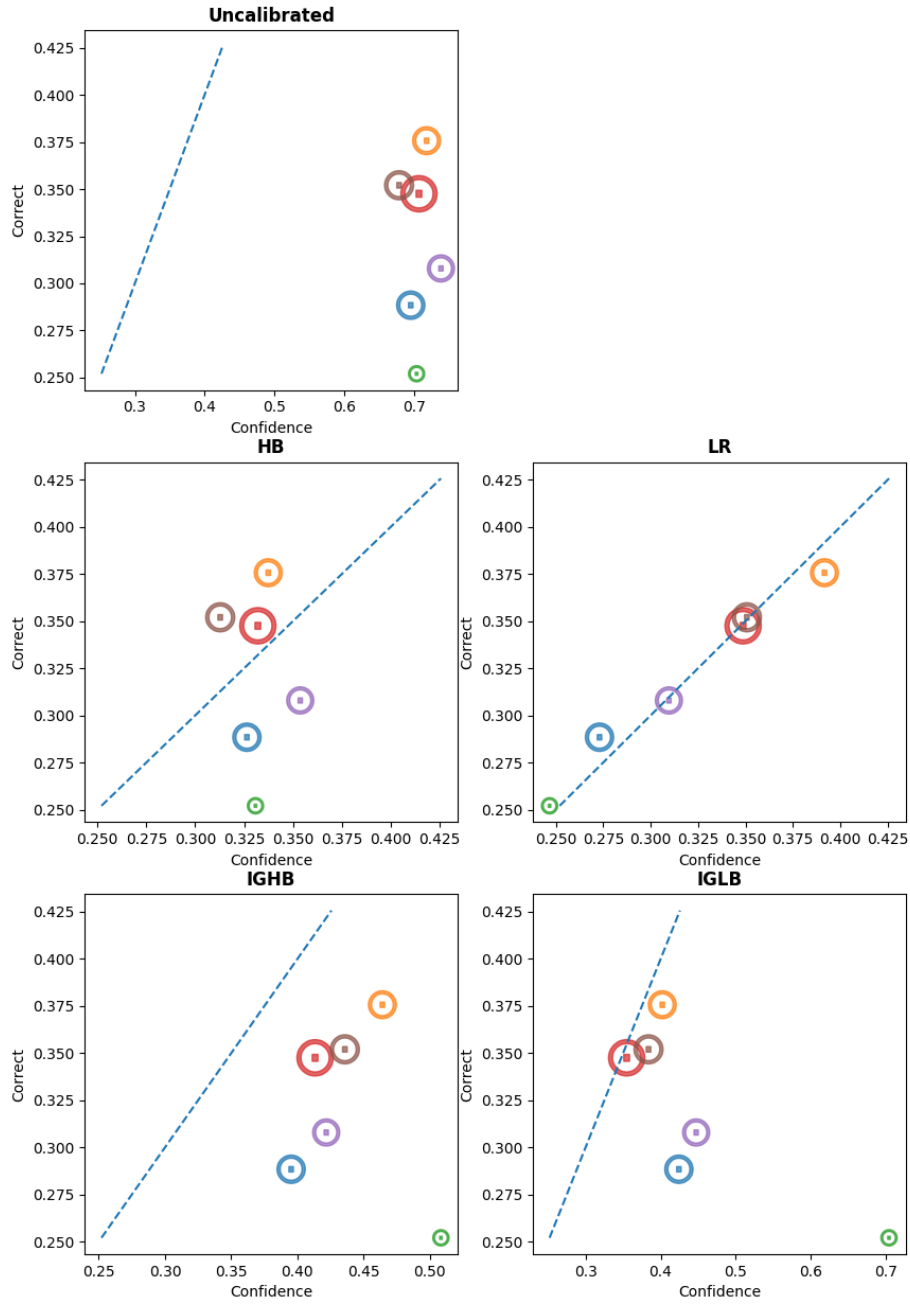


Figure 4.7: Group calibration for the uncalibrated data and then for each approach we used for calibration. Each circle stand for a group. The size of the circles indicate the amount of samples. Confidence is on the x-axis and the correctness of the group is in the y-axis.

To conclude we can say that the simple approaches histogram binning and linear regression delivered the best calibration results. And the iterative group histogram binning and iterative group linear binning approach performed worse but not quite bad. Furthermore the linear regression delivered the

best calibration for the groups.

4.3.6 Eavluator LLM

4.4 Experiments

4.4.1 Using different grouping styles

In this experiment we want to evaluate the impact of different grouping styles. We defined this styles the in the chapter 3. To recapitulate we got the simple grouping style, complexity and categories.

- The simple grouping style checks if the prompts is longer then the median prompt length, if the prompt contains the word „example“and if the code is longer than the median code.
- With the complexity grouping style we classify each program into categories of how complex the generated program is.
- The categories grouping style assigns the code groups by checking for certain keywords. For example checking for the keywords like 'integer', 'float' and a combination of 'number' and 'calculate'.

We want to answer the question: which grouping style provides better calibration?

4.4.2 Using counter groups

The next experiment deals with the usage of counter groups. Counter groups are the exact opposite of a defined group. If a sample is not in the group „examples“it is assigned to the counter group „**not** examples“. This enables us to also target the samples which are not represented in a group and adjust their probabilities.

The question arises: Does the usage of counter groups improve tha calibration results?

Chapter 5

Discussion

Sources

Literaturverzeichnis

- [1] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3675–3691, 2023.
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.
- [3] Gianluca Detommaso, Martin Bertran, Riccardo Fogliato, and Aaron Roth. Multicalibration for confidence scoring in llms, 2024.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [5] Fabrizio Le. How chatgpt is transforming the postdoc experience. *Nature*, 622:655, 2023.
- [6] T.J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976.
- [7] Ansong Ni, Pengcheng Yin, Yilun Zhao, Martin Riddell, Troy Feng, Rui Shen, Stephen Yin, Ye Liu, Semih Yavuz, Caiming Xiong, Shafiq Joty, Yingbo Zhou, Dragomir Radev, Arman Cohan, and Arman Cohan. L2ceval: Evaluating language-to-code generation capabilities of large language models. *Transactions of the Association for Computational Linguistics*, 12:1311–1329, 10 2024.
- [8] Claudio Spiess, David Gros, Kunal Suresh Pai, Michael Pradel, Md Rafiqul Islam Rabin, Amin Alipour, Susmit Jha, Prem Devanbu, and Toufique Ahmed. Calibration and correctness of language models for code. *arXiv preprint arXiv:2402.02047*, 2024.

- [9] Dennis Ulmer, Martin Gubri, Hwaran Lee, Sangdoo Yun, and Seong Oh. Calibrating large language models using their generations only. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 15440–15459, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [10] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems, CHI EA '22, New York, NY, USA, 2022. Association for Computing Machinery.
- [11] Yuvraj Virk, Premkumar Devanbu, and Toufique Ahmed. Enhancing trust in llm-generated code summaries with calibrated confidence scores. arXiv preprint arXiv:2404.19318, 2024.
- [12] Li Zhong and Zilong Wang. Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 38, pages 21841–21849, 2024.

Online-Quellen

- [13] BigCodeBench. Bigcodebench. [letzter Zugriff: 13. Jun. 2025].
- [14] Ben Boyter. Sloc cloc and code (scc). [letzter Zugriff: 6. Jun. 2025].
- [15] Thomas McCabe Jr. Software quality metrics to identify risk. [letzter Zugriff: 6. Jun. 2025].

Appendix A

Prompts

A.1 Evaluator LLM

"Answer as short as you can. Here is a snippet of generated code consisting of the prompt, generated code and test cases: {code}"

Verbalized quantitative prompt: "Please provide your confidence in the correctness of the provided code only in percent (0-100 %): "

Verbalized qualitative prompt: "Please provide your confidence in the correctness of the provided code only as one of {Qualitative scale:}; "

Qualitative scale: "
{
"Very low": 0,
"Low": 0.3,
"Somewhat low": 0.45,
"Medium": 0.5,
"Somewhat high": 0.65,
"High": 0.7,
"Very high": 1,
}"